

Predicting song popularity

April 2026

Dziyana Melnikava: Data Engineer
Rail Sabirov: Data Scientist
Dzhamilia Fatkullina: ML Specialist
Ruslan Gatiatullin: Technical Writer

Contents

1	Introduction	3
1.1	Business Objectives	3
2	Data Description	3
2.1	Data Characteristics	3
3	Architecture of data pipeline	4
4	Data preparation	4
4.1	ER diagram	4
4.2	Samples from the database	4
4.3	Hive tables and preparing the data for analysis	5
4.3.1	Overview	5
4.3.2	Target Variable	5
4.3.3	Feature included in the cleaned dataset	6
4.3.4	Removed features	6
4.3.5	Encodings	6
4.3.6	Added columns	6
4.3.7	Final dataset schema	7
4.3.8	Creating Hive tables	8
5	Data analysis	9
5.1	Analysis results	9
5.2	Charts	10
5.3	Interpretation	12
5.3.1	Average first-month streams by release month	12
5.3.2	Average first-month streams by release weekday	12
5.3.3	Average first-month streams by number of markets	12
5.3.4	Average first-month streams by artist's number of tracks in Top-200	13
5.3.5	Average first-month streams by artist historical strength	13
5.3.6	Average first-month streams by audio profile	13
6	ML modeling	13
6.1	Feature extraction and data preprocessing	13
6.2	Training and fine-tuning	14
6.3	Evaluation	14
7	Data presentation	14
7.1	The description of the dashboard	14
7.2	Description of each chart	15
7.3	Your findings	16
8	Conclusion	16
8.1	Summary of the report	16
9	Reflections on own work	17
9.1	Challenges and difficulties	17
9.2	Recommendations	17
9.3	The table of contributions of each team member	19

1 Introduction

1.1 Business Objectives

- Primary business objective: To predict the popularity of a song before its release based on lyrics, genre, loudness etc.
- Secondary business objective: Analyze the importance of each factor in song's popularity.

2 Data Description

This is a complete dataset of all the "Top 200" and "Viral 50" charts published globally by Spotify since January 1, 2019. Dataset source:
<https://www.kaggle.com/datasets/sunnykakar/spotify-charts-all-audio-data>

2.1 Data Characteristics

- 29 columns: 15 floats, 8 strings, 3 category, 2 integers, 1 date
- 26.2m rows
- 27.1 GB in size
- Target column: number of streams in the first month after release (received by summing the 'streams' column across all countries)

Table 1: Full column description

Column Name	Description	Type
title	Song title	string
rank	Chart position (e.g., 1 = top)	int
date	Date of the chart entry	date (YYYY-MM-DD)
artist	Name of the performing artist	string
url	Spotify URL of the track	string
region	Country/region code (e.g., US, GB)	string
chart	Chart type (e.g., top200, viral50)	string
trend	Trend indicator (e.g., UP, DOWN, NEW)	string
streams	Number of streams in the tracking period	int
track_id	Unique Spotify track identifier	string
album	Album name of the track	string
popularity	Spotify popularity score (0–100)	int
duration_ms	Track duration in milliseconds	int
explicit	Whether the track contains explicit content	boolean
release_date	Original release date of the track	date (YYYY-MM-DD)
available_markets	Comma-separated list of ISO country codes where the track is available	string
af_danceability	Danceability score (0–1) from Spotify audio features	double
af_energy	Energy score (0–1)	double
af_key	Key of the track (Pitch class notation, e.g., 0 = C, 1 = C#, ..., 11 = B)	int
af_loudness	Overall loudness in decibels (dB)	double

af_mode	Modality (0 = minor, 1 = major)	int
af_speechiness	Presence of spoken words (0–1)	double
af_acousticness	Acousticness score (0–1)	double
af_instrumentalness	Instrumentalness score (0–1; 1 = no vocals)	double
af_liveness	Liveness score (0–1; presence of audience)	double
af_valence	Musical positivity/valence (0–1)	double
af_tempo	Estimated tempo in beats per minute (BPM)	double
af_time_signature	Time signature (e.g., 3 for 3/4, 4 for 4/4)	int

3 Architecture of data pipeline

This section describes the input and the output for each stage.

1. Stage I - Data collection and Ingestion

- Input: raw dataset, business objective
- Output: SQL tables with a preprocessed data subset (imported to HDFS serialized in parquet format and compressed using Snappy codec.)

2. Stage II - Data Storage/Preparation & EDA

- Input: SQL tables with a preprocessed data subset
- Output: Hive database with tables, data insights and graphs for them, scripts for automating Hive setup and launching queries for EDA

3. Stage III - Predictive Data Analytics

- Input: Hive tables, completed EDA
- Output: 2 ML models for predicting the target column, their evaluation and comparison with each other

4. Stage IV - Presentation & Delivery

- Input: business objective, completed EDA and PDA, 2 ML models with evaluated results
- Output: web dashboard with data description and insights, external Hive tables and chats for ML models, scripts for automating everything before

4 Data preparation

4.1 ER diagram

We only have a single table, so ER diagram is not needed.

4.2 Samples from the database

Table 2: Samples from the initial dataset

Feature	Song 1: Chantaje (feat. Maluma)	Song 2: Vente Pa' Ca (feat. Maluma)
title	Chantaje (feat. Maluma)	Vente Pa' Ca (feat. Maluma)
rank	1	2
date	2017-01-01	2017-01-01
artist	Shakira	Ricky Martin
url	https://open.spotify.com/track/6mICuAdrwEjh6Y6lroV2Kg	https://open.spotify.com/track/7DM4BPas7uofFul3ywMe46
region	Argentina	Argentina
chart	top200	top200
trend	SAME_POSITION	MOVE_UP
streams	253019.0	223988.0
track_id	6mICuAdrwEjh6Y6lroV2Kg	7DM4BPas7uofFul3ywMe46
album	El Dorado	Vente Pa' Ca (feat. Maluma)
popularity	78.0	72.0
duration_ms	195840.0	259195.0
explicit	False	False
release_date	2017-05-26	2016-09-22
available_markets	['AR', 'AU', ..., 'XK'] (125+ markets)	['AR', 'AU', ..., 'XK'] (identical list)
af_danceability	0.852	0.663
af_energy	0.773	0.92
af_key	8.0	11.0
af_loudness	-2.921	-4.07
af_mode	0.0	0.0
af_speechiness	0.0776	0.226
af_acousticness	0.187	0.00431
af_instrumentalness	3.05e-05	1.69e-05
af_liveness	0.159	0.101
af_valence	0.907	0.533
af_tempo	102.034	99.935
af_time_signature	4.0	4.0

4.3 Hive tables and preparing the data for analysis

4.3.1 Overview

This preprocessing pipeline transforms the raw Spotify Charts dataset into a clean, structured dataset. The output dataset contains **30 columns**: 29 features and 1 target variable.

We have decided to work with .parquet files because fast reading is more important for us that constantly adding new data.

4.3.2 Target Variable

The business goal is to predict a song's popularity **before release**. The natural proxy for post-release popularity is the number of streams accumulated during the first 30 days.

Calculation:

- Filter rows where 'chart = 'top200'' (viral50 has NULL streams)

- Compute ‘days_since_release = date - release_date’
- Keep only days 0 to 29 after release
- Group by ‘track_id’ and sum ‘streams’

4.3.3 Feature included in the cleaned dataset

Table 3: Feature included in the cleaned dataset

Category	Features	Count	Justification
Audio features	af_danceability, af_energy, key_sin, key_cos, af_loudness, af_mode, af_speechiness, af_acousticness, af_instrumentalness, af_liveness, af_valence, af_tempo, af_time_signature	13	Available before release, Spotify’s audio analysis has proven predictive power for popularity
Track metadata	duration_ms, explicit	2	Known before release
Seasonal features	release_month, release_day_of_week, release_week_of_year	3	Extracted from release_date, capture release timing effects (e.g., Friday releases, holiday seasons)
Market availability	num_markets + 8 binary flags (US, GB, DE, JP, BR, FR, CA, AU)	9	Derived from available_markets
Artist history	artist_prior_tracks_count, artist_avg_past_streams	2	Computed from previous tracks of the same artist, established artists tend to perform better

4.3.4 Removed features

Table 4: Columns removed during preprocessing

Column	Reason for Removal
region	Post-release information
rank, trend, date	Post-release chart positions
chart	Post-release (top200/viral50)
streams	Used only to build target variable
popularity	Post-release metric
url, track_id, album	Non-informative or identifiers
available_markets (raw)	Replaced by derived features

4.3.5 Encodings

We use cyclical Encoding for ‘af_key’ because musical key is cyclical ($C \rightarrow C\# \rightarrow \dots \rightarrow B \rightarrow C$). $key_sin = \sin(2\pi * key/12)$, $key_cos = \cos(2\pi * key/12)$

4.3.6 Added columns

- ‘artist_prior_tracks_count’ – Number of previous tracks by the same artist
- ‘artist_avg_past_streams’ – Average ‘streams_in_the_first_month’ of previous tracks

- Binary flags for top 8 markets since they contribute to the absolute majority of the streams: US, GB, DE, JP, BR, FR, CA, AU.

4.3.7 Final dataset schema

Table 5: Final Dataset Schema

Column	Type	Description
af_danceability	double	Danceability score (0–1)
af_energy	double	Energy score (0–1)
key_sin	double	Cyclical encoding of key (sine)
key_cos	double	Cyclical encoding of key (cosine)
af_loudness	double	Loudness (dB)
af_mode	int	Mode (0 = minor, 1 = major)
af_speechiness	double	Speechiness score (0–1)
af_acousticness	double	Acousticness score (0–1)
af_instrumentalness	double	Instrumentalness score (0–1)
af_liveness	double	Liveness score (0–1)
af_valence	double	Valence (musical positivity, 0–1)
af_tempo	double	Tempo (BPM)
af_time_signature	double	Time signature (e.g., 4 for 4/4)
duration_ms	double	Track duration in milliseconds
explicit	int	1 = explicit, 0 = not explicit
release_month	int	Month of release (1–12)
release_day_of_week	int	Day of week (1 = Monday, 7 = Sunday)
release_week_of_year	int	Week number (1–53)
num_markets	int	Number of countries where track is available
is_market_us	int	1 = available in US
is_market_gb	int	1 = available in GB
is_market_de	int	1 = available in DE
is_market_jp	int	1 = available in JP
is_market_br	int	1 = available in BR
is_market_fr	int	1 = available in FR
is_market_ca	int	1 = available in CA
is_market_au	int	1 = available in AU
artist_prior_tracks_count	int	Number of previous tracks by the same artist
artist_avg_past_streams	double	Average first-month streams of previous tracks
streams_in_the_first_month	double	Total streams in first 30 days (target variable)

4.3.8 Creating Hive tables

The screenshot displays a JupyterLab environment with a terminal window showing the execution of Hive commands to create a table named `songs_part_bucket`. The terminal output includes the following information:

```
INFO : Compiling command(queryId=hive_20260419172619_340bba4b-aa27-4bf3-a109-e6c0bb229644): DESCRIBE FORMATTED songs_part_bucket
INFO : Semantic Analysis Completed (metaData = false)
INFO : Returning Hive schema: Schema(fieldsSchemas([FieldSchema(name=col_name, type=string, comment=from deserializer), FieldSchema(name=data_type, type=string, comment=from deserializer), FieldSchema(name=comment, type=string, comment=from deserializer)], properties=null))
INFO : EXPLAIN output for queryId hive_20260419172619_340bba4b-aa27-4bf3-a109-e6c0bb229644 : STAGE DEPENDENCIES:
Stage-0 is a root stage [DDL]
Stage-1 depends on stages: Stage-0 [FETCH]

STAGE PLANS:
Stage: Stage-0
Describe Table Operator:
Describe Table
result file: file:/tmp/hive/5484357f-ab72-4827-ab63-881fb3f7320e/hive_2026-04-19_17-26-19_461_6585386325892760126-15157/-local-10000
table: songs_part_bucket

Stage: Stage-1
Fetch Operator
limit: -1
Processor Tree:
ListSink

INFO : Completed compiling command(queryId=hive_20260419172619_340bba4b-aa27-4bf3-a109-e6c0bb229644): Time taken: 0.041 seconds
INFO : Executing command(queryId=hive_20260419172619_340bba4b-aa27-4bf3-a109-e6c0bb229644): DESCRIBE FORMATTED songs_part_bucket
INFO : Starting task [Stage-1(DDL)] in serial mode
INFO : Completed executing command(queryId=hive_20260419172619_340bba4b-aa27-4bf3-a109-e6c0bb229644): Time taken: 0.043 seconds
INFO : OK
```

Below the terminal output, the table schema is displayed in a table format:

col_name	data_type	comment
# col_name		
id	bigint	
af_danceability	double	
af_energy	double	
key_sls	double	
key_cos	double	
af_loudness	double	
af_mode	int	
af_speechiness	double	
af_acousticness	double	
af_instrumentalness	double	
af_liveness	double	
af_valence	double	
af_tempo	double	
af_time_signature	double	
duration_ms	double	
explicit	int	
release_day_of_week	int	
release_week_of_year	int	
num_markets	int	
is_market_us	int	
is_market_gb	int	
is_market_de	int	
is_market_jp	int	
is_market_br	int	
is_market_fr	int	
is_market_ca	int	
is_market_aus	int	

Figure 1: Hive table

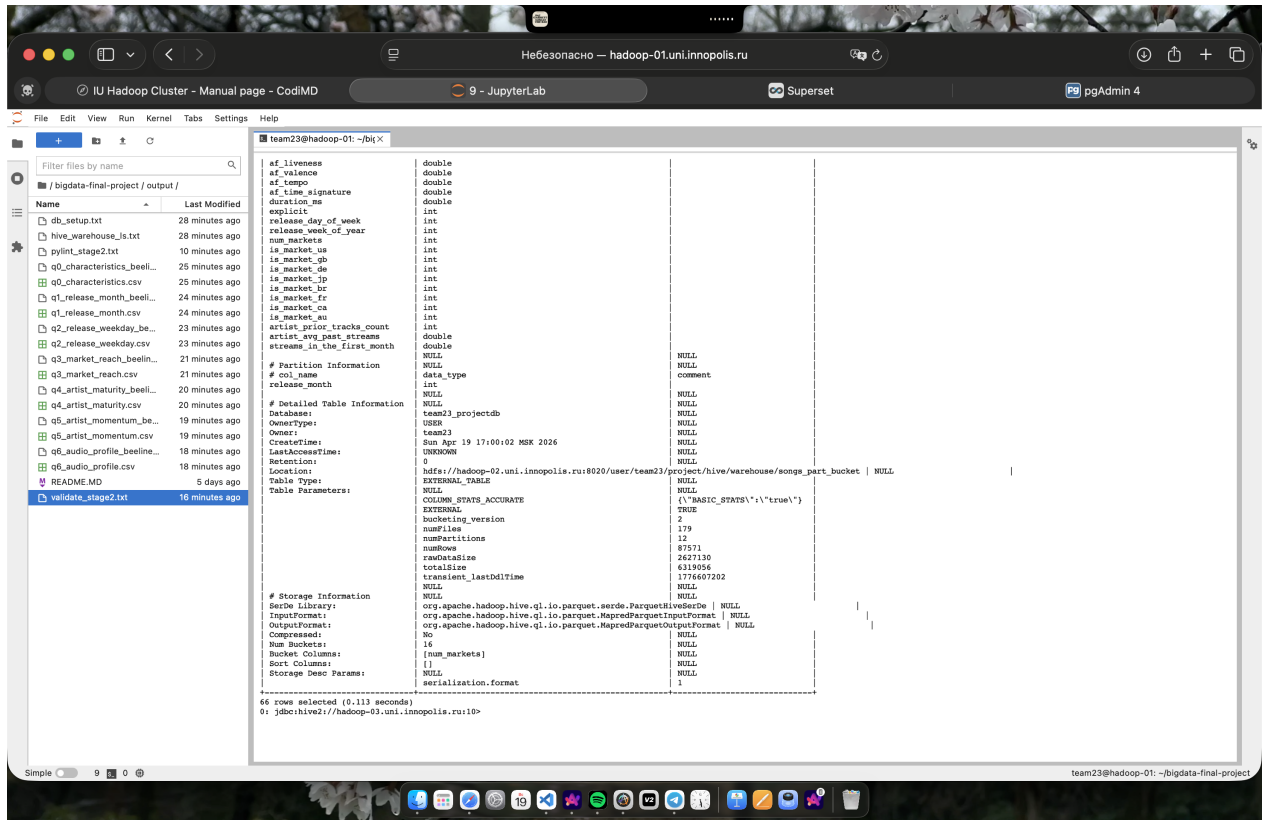


Figure 2: Hive table

5 Data analysis

5.1 Analysis results

We performed EDA on the Spotify dataset using Hive as our analytical engine. First, we optimized the data for querying by creating an external Hive table with partitioning on `release_month` and bucketing on `num_markets`. We then loaded the data into this optimized table, which allowed us to filter efficiently and improve query performance for all subsequent analyses.

We wrote six HiveQL queries to investigate factors that could influence a track's first-month streaming performance. Specifically, we examined release timing (month and weekday), number of targeted markets, artist maturity (number of prior tracks), artist momentum (average past streams), and audio profile (combinations of danceability and energy). For each query, we have made clear visualizations along with their interpretations.

5.2 Charts

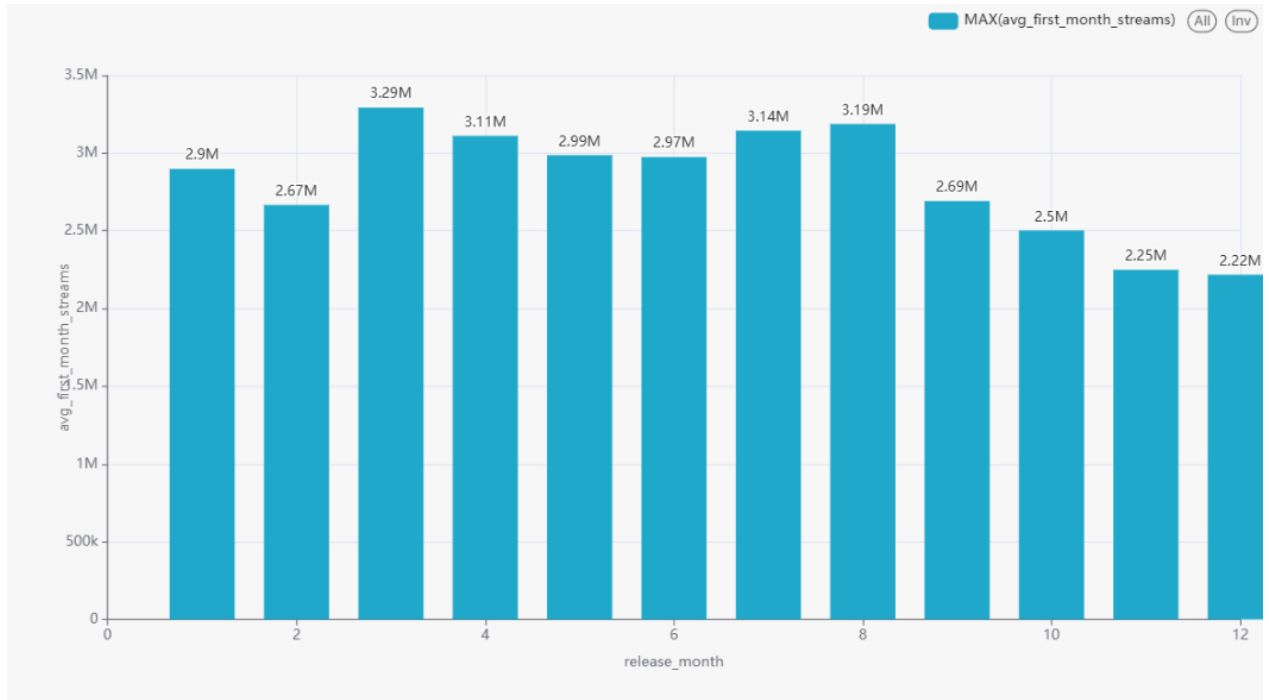


Figure 3: Average first-month streams by release month

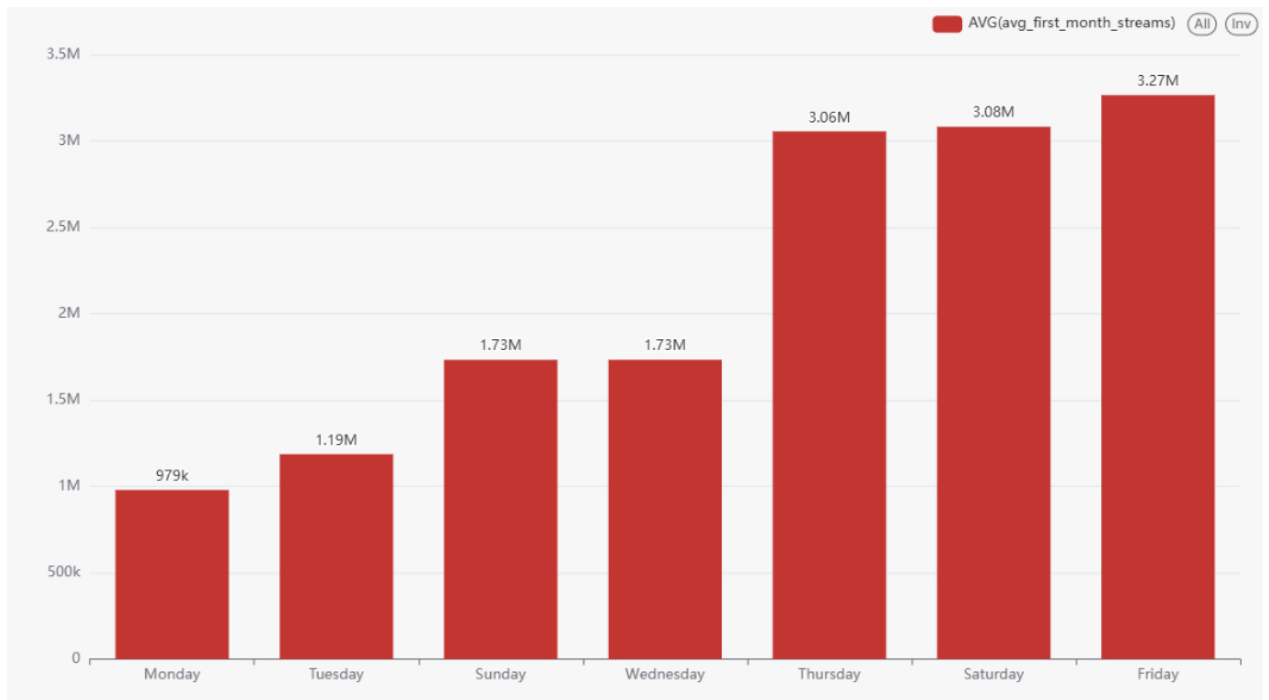


Figure 4: Average first-month streams by release weekday

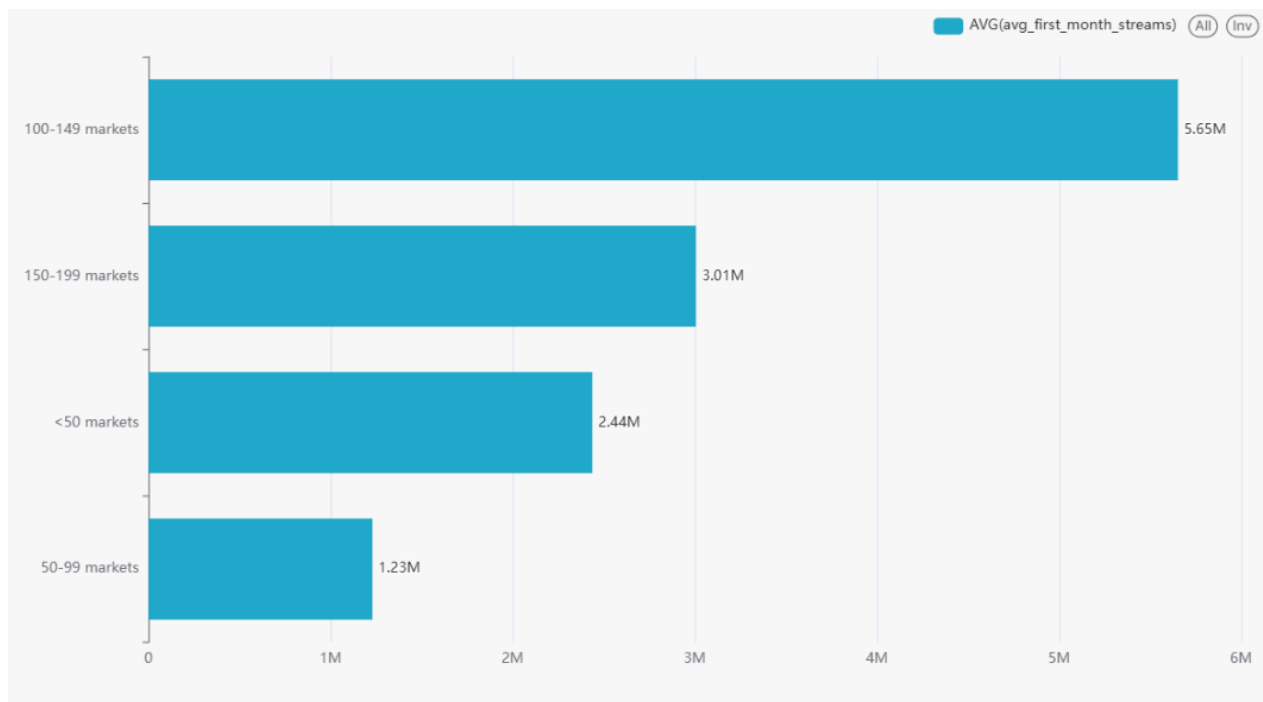


Figure 5: Average first-month streams by number of markets

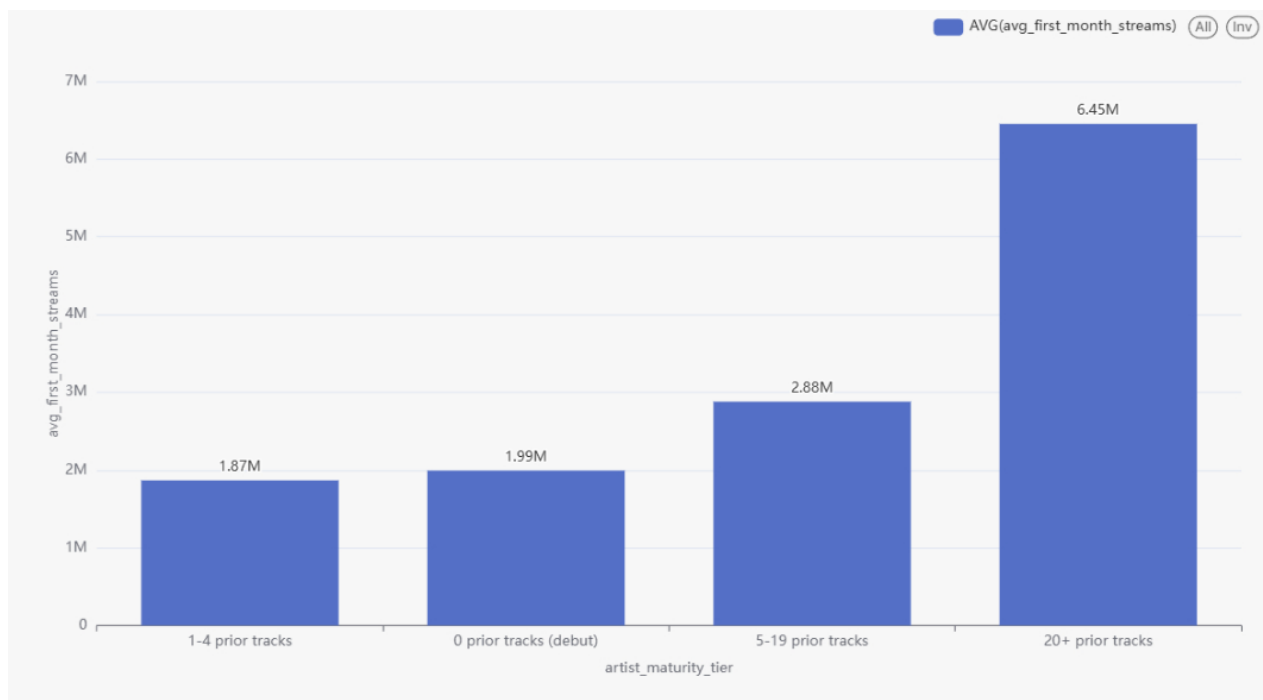


Figure 6: Average first-month streams by artist's number of tracks in Top-200

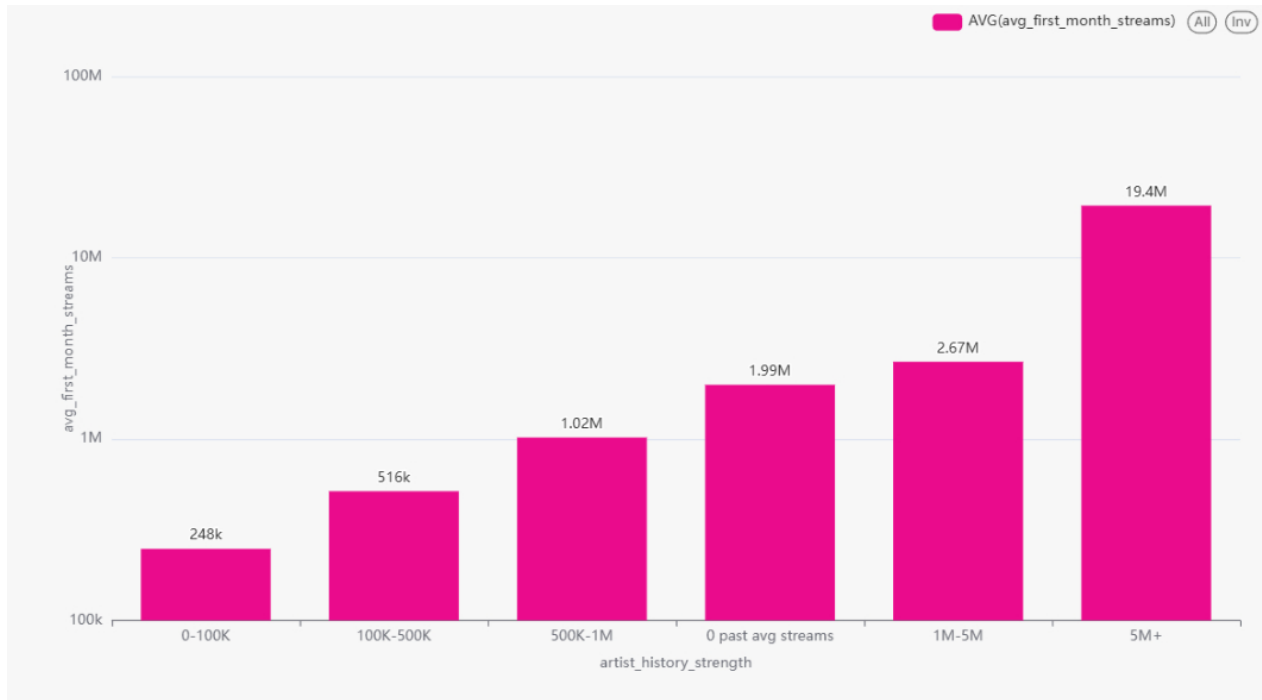


Figure 7: Average first-month streams by artist historical strength

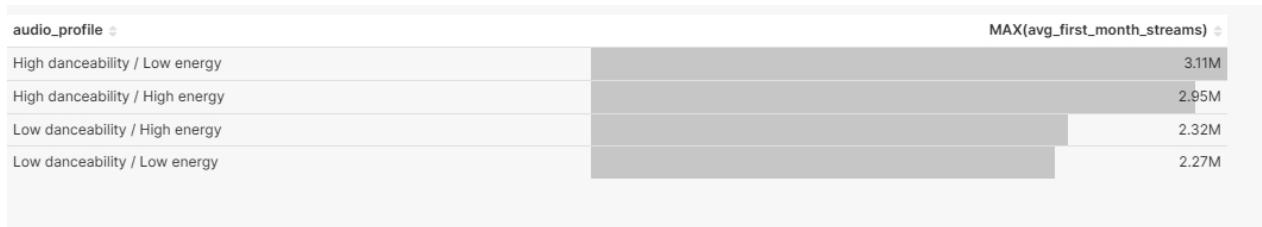


Figure 8: Average first-month streams by audio profile

5.3 Interpretation

5.3.1 Average first-month streams by release month

Figure 3 shows the amount of streams the song gets in the first month weekly relates to the release month. According to that graph, March is the best month to release a song with an average of 3.29M first-month streams, while December is the worst with 2.22M average first-month streams.

5.3.2 Average first-month streams by release weekday

Figure 4 shows that the release day of the week may have some impact on the song's performance. It is better to release a song on Thursday, Saturday or Friday. On average that would lead to x2-x3 better result than a release on any other day of the week.

5.3.3 Average first-month streams by number of markets

Figure 5 show that songs that release in 100-149 markets typically perform the best within the first month. On the other hand, songs that release in 50-99 markets perform way worse than all other categories, even

than <50 markets category, which is surprising.

5.3.4 Average first-month streams by artist’s number of tracks in Top-200

Figure 6 shows that songs by musicians with > 20 previous tracks perform 2.2-3.2x better than songs by musicians with < 20 previous tracks. That makes sense because those musicians are typically more experienced and already have a bigger audience.

5.3.5 Average first-month streams by artist historical strength

Figure 7 shows how a song’s success depends on the track record of its artist. The y-axis uses a logarithmic scale. The figure demonstrates that artists with a history of higher average streams consistently achieve greater success with new releases. Interestingly, for artists whose historical average streams fall between 0 and 100,000, new songs tend to far outperform previous ones—likely because these musicians are breaking through with their first major hit.

5.3.6 Average first-month streams by audio profile

Figure 8 shows that high dance-ability and low energy songs typically perform the best while low dance-ability and low energy songs typically perform the worst. Nevertheless, audio profile does not matter that much since the difference between best and worst profiles is only 37%.

6 ML modeling

For the ML modeling we have decided to use Linear regression as the first model and Gradient Boosted Tree Regressor (GBTRegressor) as the second.

6.1 Feature extraction and data preprocessing

Here are key decisions on feature extraction and data preprocessing:

1. **Both models predict $\log_{10}(\text{streams in the first month})$** — the log-transformed total streams a song accumulates in its first 30 days after release. The target was transformed because the raw stream counts are extremely right-skewed (min $\approx 1\text{K}$, max $\approx 612\text{M}$, mean $\approx 2.8\text{M}$), which makes untransformed regression unstable.
2. Log-transform was also applied to `artist_avg_past_streams` for the same reason.
3. For linear regression we have applied `StandardScaler` fitted on training data because the feature scale is very different. We don’t use it for `GBTRegressor` since tree split thresholds are invariant to monotonic feature transformations.
4. The data is split 80/20 for training and testing with a consideration of time. This ensures no data leakage.
5. Cyclical encoding is used for month and day of the week.
6. Other data transformations were done earlier and are described in the Data preparation section.

6.2 Training and fine-tuning

Linear regression is trained for minimizing MSE. It uses grid search and 3-fold cross validation to find the best hyperparameters for L1/L2 proportion in the regularization part (elasticNetParam) and regularization coefficient (regParam).

GBTRegressor is also trained for minimizing MSE. We have additionally changed subsamplingRate to 0.8 to speed up the training. Here we also use grid search and 3-fold cross validation to find optimal max tree depth (maxDepth) and learning rate (step_size).

6.3 Evaluation

Table 6: Test set evaluation on a log1p-transformed target.

Metric / Model	RMSE	MAE	R^2
Global mean baseline	2.38	1.92	0.00
Linear Regression	2.17	1.75	0.17
GBT Regressor	1.95	1.45	0.37

Unsurprisingly GBTRegressor performs a lot better than linear regression.

An R^2 of 0.37 means that 37% of variance in the data can be explained by our model. This is the realistic result for pre-release stream prediction given the available features. The unpredictable portion of the variance — Spotify playlist placement, editorial features, social media virality (especially TikTok) — is not observable before a song’s release and therefore cannot be modeled. What the model does capture is the structural component.

As for MAE, taking its exponent gives us 8.75 and 4.26 for LR and GTBRegressor respectively. This means that on average our models can make a mistake by a factor of 8.75 and 4.26. Again, this is quite high but expected.

Compared to the baseline of just predicting the mean, our models show substantial improvement, meaning they have learned useful patterns in the data.

7 Data presentation

7.1 The description of the dashboard

The dashboard contains 3 sections: Dataset overview, EDA insights, and Spark ML modeling results.

Dataset overview section presents the description of the dataset: its characteristics, and a few samples.

EDA insights section shows the graph from the Data Analysis section in the report 5.

Spark ML modeling section contains charts about model results and training.

7.2 Description of each chart

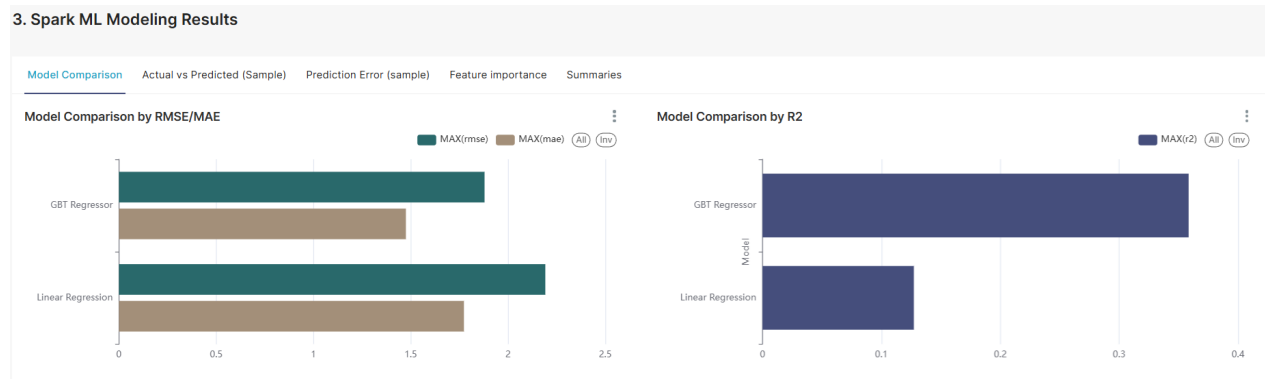


Figure 9: Spark ML metrics

This chart describes models' metrics: RMSE, MAE, R^2 .

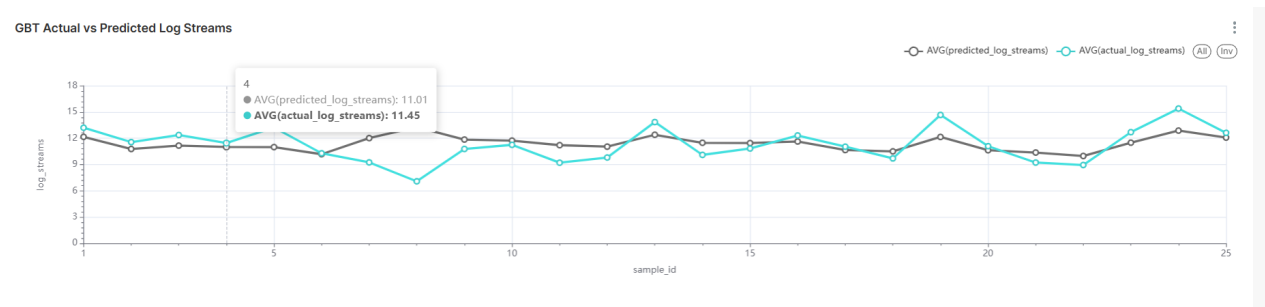


Figure 10: Spark ML predictions

This chart shows GBTRRegressor prediction vs actual target.

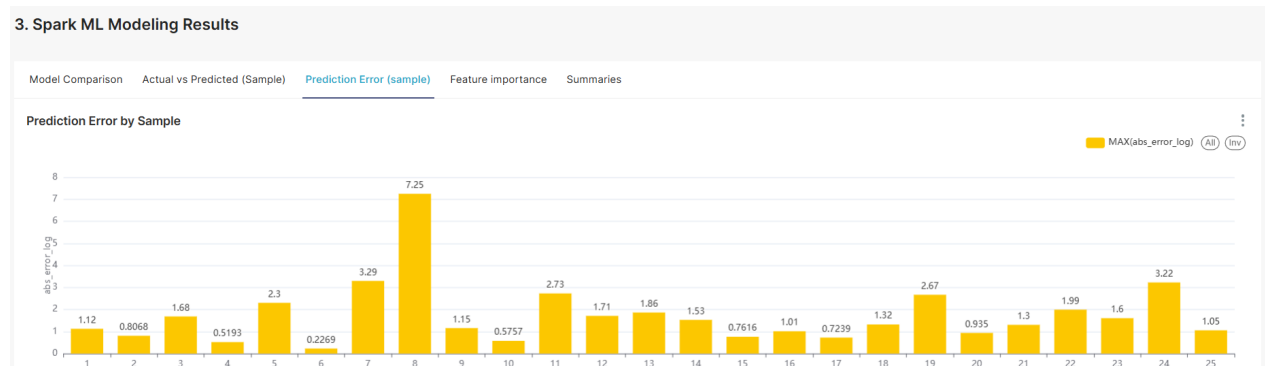


Figure 11: Prediction errors

This chart shows errors from the predictions in the previous chart.

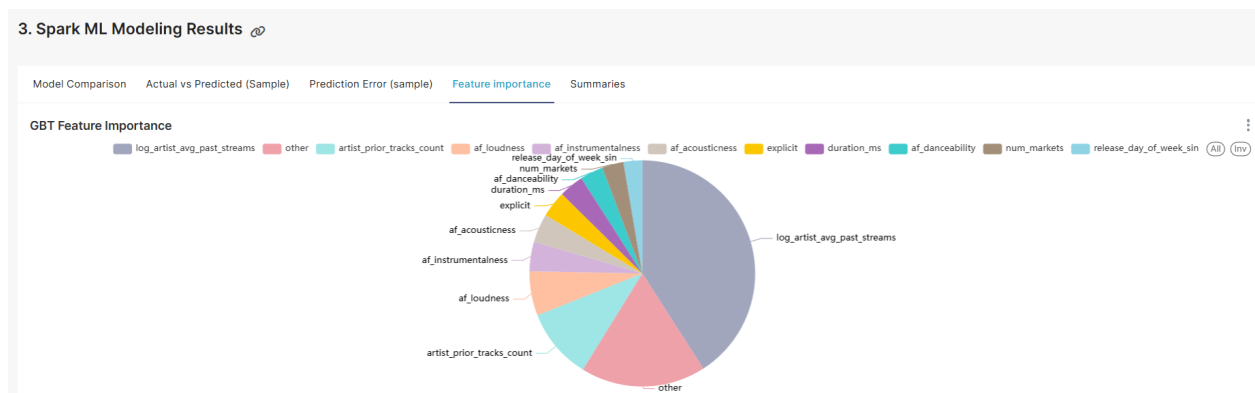


Figure 12: Feature importance from GBTRRegressor

This chart shows the importance of each feature in GBTRRegressor's predictions.

3. Spark ML Modeling Results

Model Comparison Actual vs Predicted (Sample) Prediction Error (sample) Feature Importance **Summaries**

Feature Extraction Summary

feature_group	feature_name	description	used_in_model
Audio features	af_danceability	Danceability score from Spotify audio analysis	yes
Audio features	af_energy	Energy score from Spotify audio analysis	yes
Audio features	key_sin	Sine encoding of musical key	yes
Audio features	key_cos	Cosine encoding of musical key	yes
Audio features	af_loudness	Track loudness in decibels	yes
Audio features	af_mode	Musical mode flag	yes
Audio features	af_speechiness	Speechiness score	yes
Audio features	af_acousticness	Acousticness score	yes
Audio features	af_instrumentalness	Instrumentalness score	yes
Audio features	af_liveness	Liveness score	yes
Audio features	af_valence	Musical positivity score	yes

Hyperparameter Tuning Summary

model	hyperparameter	searched_values	best_value	reason
Linear Regression	regParam	0.001; 0.01; 0.1	0.001	Controls regularization strength
Linear Regression	elasticNetParam	0.0; 0.5; 1.0	0.0	Controls the Ridge and Lasso regularization mix
GBT Regressor	maxDepth	3; 5	5	Controls the depth and complexity of each tree
GBT Regressor	stepSize	0.05; 0.1	0.1	Controls the boosting learning rate

Figure 13: Used features

This chart shows the features included in the dataset for training the model.

7.3 Your findings

The most important predictor of song's success is the average number of streams on songs for this musician, which is expected.

Second best predictor is the track count.

Third factor is song's loudness.

8 Conclusion

8.1 Summary of the report

In this project, we developed a complete data pipeline to predict the first-month streaming popularity of songs before their release using the Spotify Charts dataset. The pipeline consisted of four stages: data inges-

tion and storage (PostgreSQL, HDFS), data preparation and exploratory analysis (Hive, HiveQL), predictive modelling (Spark ML), and final presentation (Apache Superset dashboard).

Our exploratory data analysis revealed several actionable insights. March was identified as the most favourable release month, while Thursday, Friday and Saturday were the best weekdays for launching a track. Releasing a song in 100–149 markets was associated with the strongest early performance. Artist experience proved highly influential: musicians with more than 20 prior tracks in the Top-200 achieved 2.2–3.2 times higher first-month streams than less experienced artists. Similarly, artists with a history of higher average streams consistently outperformed others. Audio profile had a smaller but noticeable effect: songs with high danceability and low energy performed best, though the difference between the best and worst profiles was only 37%.

For predictive modelling, we compared a Linear Regression baseline with a Gradient Boosted Tree Regressor. Because the target variable (first-month streams) was extremely right-skewed, we modelled $\log(\text{streams} + 1)$ and also applied a log transformation to `artist_avg_past_streams`. Using an 80/20 time-aware split, the GBT Regressor achieved an R^2 of 0.37, an RMSE of 1.95 and an MAE of 1.45 on the log-transformed scale, substantially outperforming Linear Regression ($R^2 = 0.17$, RMSE = 2.17, MAE = 1.75) and the global mean baseline ($R^2 = 0.00$). The remaining unexplained variance is attributable to factors that are not observable before a song’s release, such as playlist placement, editorial features and social media virality (e.g., TikTok).

Overall, the project successfully delivered a functional data engineering pipeline, an interactive dashboard, and two trained ML models that capture a meaningful structural component of song popularity. The work also highlighted practical challenges, including HDFS memory constraints and network restrictions, and led to recommendations such as early cross-validation of code to avoid late-stage mistakes.

9 Reflections on own work

Overall, we have showed great team work with each team member being responsible and competent about their role.

9.1 Challenges and difficulties

One of the main challenges was related to running out of memory quota in HDFS. It was solved when the quota got increased to 32 GB and we have set the replication factor to 1.

Additionally, the need to use exclusively university network for the cluster access was a significant inconvenience for us.

9.2 Recommendations

Our main recommendation is to double check each other’s work since we have found a few significant mistakes in our code late in the project.

9.3 The table of contributions of each team member

Table 7: Project tasks and contributions

Project Tasks	Task description	Diana Melnikava	Rail Sabirov	Dzhamilia Fatkul-lina	Ruslan Gatiat-ullin	deliverables	Avg hours spent
Analyze the business problem	Analyze the business problem, prepare business objectives	0%	0%	0%	100%	Business objectives	2.0
Build databases for data storage	Build databases for data storage and Ingest the data into HDFS	100%	0%	0%	0%	PostgreSQL table	4.0
Data cleaning and transformations	Clean the data, determine which transformations to use, apply them	100%	0%	0%	0%	Preprocessed data	3.0
Create Hive tables	Create Hive tables	0%	100%	0%	0%	Hive tables	4.0
EDA	Perform EDA using HiveQL	0%	100%	0%	0%	6 charts that provide insights about the data	4.0
Build Spark ML models	Build Spark ML models, train and evaluate the models, do hyperparameter tuning and Grid search	0%	0%	100%	0%	2 ML ready-to-use models with their evaluation	7.0
Build a web dashboard in Apache Superset and publish it	Build a web dashboard in Apache Superset and publish it	0%	70%	30%	0%	Apache Superset Dashboard about models' performance	4.0
Prepare report	Prepare this report	0%	0%	0%	100%	A PDF report about the whole project	6.0
Prepare presentation	Prepare final project presentation	0%	0%	0%	100%	Final project presentation	2.0